

Prompt Engineering & Context Engineering for Agents

Master the art of crafting precise prompts and engineering reliable AI context



Learning Outcome

By the end of this session, you will be able to write high-quality prompts for any business task, configure reliable system prompts for agents, and apply context engineering to eliminate hallucination and improve output accuracy.



Prompt Anatomy

5 components



Techniques

4 methods



System Prompts

Architecture



Context

Engineering



Hallucination

Prevention



Iteration

Optimization

⚠️ The Problem — Why This Matters Now

Organizations are losing productivity and facing risks from poorly engineered AI interactions



Inconsistent outputs waste hours

Teams spend 40% of time fixing AI-generated content



Hallucinations risk brand & compliance

Wrong information damages credibility and legal standing



Vague prompts create rework loops

Multiple iterations needed to get acceptable results



Agents misapply tools without guardrails

Lack of constraints leads to incorrect tool usage



Stakeholders lose trust, adoption stalls

Poor AI experiences reduce organizational buy-in

AI Without Proper Engineering



Like navigating without a map, unengineered prompts lead to unpredictable outcomes and wasted resources

73%

Projects delayed

\$2.3M

Annual cost

45%

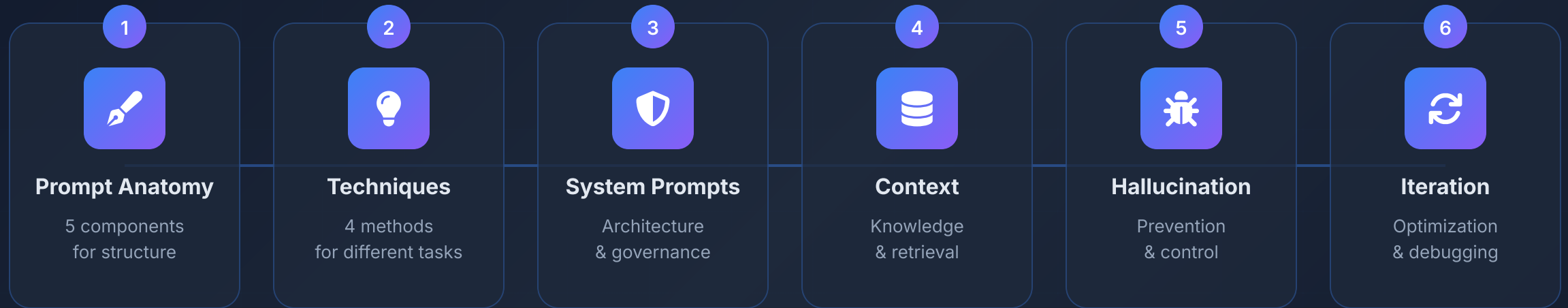
Error rate



This is costing your organization productivity and credibility

Learning Journey — Today's Roadmap

Six key sub-topics to master prompt engineering and context engineering



Key Insight: Start with simple concepts, build complexity progressively. Each sub-topic builds on the previous one to create a complete understanding of prompt engineering.



Anatomy of a Great Prompt — Blueprint

Master the 5 essential components that make prompts effective and reliable

Prompt Architecture

● Active



ROLE

Expert Financial Analyst



CONTEXT

Q3 earnings report analysis



TASK

Summarize key metrics



FORMAT

Bullet points, 5 items



CONSTRAINTS

Max 100 words, no jargon

</> Generated Prompt

```
You are an expert financial analyst.  
Analyze the Q3 earnings report.  
Summarize key metrics in 5 bullet  
points, max 100 words, no jargon.
```

Component Breakdown



Role

Sets expertise, tone, behavior



Context

Background, data, history



Task

Clear, unambiguous action



Format

Output structure & style



Constraints

Limits & guardrails



Analogy

Think of a prompt like a job description: Role = job title, Context = company background, Task = specific duties, Format = deliverable template, Constraints = company policies.

Before vs After — Applying the Blueprint

See the dramatic improvement when all 5 components are properly applied

Before: Weak Prompt

Vague & Unstructured

~~Write a follow-up email to a prospect
about our product.~~

~~Make it sound professional.~~

 Missing: Role, Context, Format, Constraints



After: Engineered Prompt

Complete & Structured

ROLE: You are a sales consultant
CONTEXT: Following up on demo with Acme Corp
TASK: Write a follow-up email
FORMAT: 3 paragraphs, bullet points
CONSTRAINTS: Max 150 words, professional tone

 Role defined

 Context added

 Format specified



Key Insight: The engineered prompt includes all 5 components, resulting in a focused, actionable email that requires minimal revision.

 2 min read



Thought Challenge — Fix Your Last Prompt

Look at the last prompt you gave an AI tool. Which of the 5 components were missing?



Component Checklist



Role

Did you specify who the AI should be? (expert, assistant, analyst)



Context

Did you provide background, data, or situation?



Task

Is the action clear, unambiguous, and verb-first?



Format

Did you specify output structure? (JSON, table, paragraph)



Constraints

Are there limits on length, tone, or topics?



Reflection Prompt

"Look at the last prompt you gave an AI tool. Which of the 5 components were missing? What would the output have looked like if you had included them?"

Think: How could you improve your next prompt using the 5-component framework?

Rewrite
Your Prompt

Predict
Output

Share
With Partner



Prompting Techniques — 2x2 Overview

Four methods for different task complexity levels and use cases



Zero-Shot

No examples given

Model uses training knowledge directly. Best for simple, well-defined tasks.

Complexity: ● ● ●

✓ Simple tasks



Few-Shot

2-5 examples provided

Model learns the pattern from examples. Best for tasks requiring specific format.

Complexity: ● ● ●

✓ Pattern learning



Chain-of-Thought

Step-by-step reasoning

Prompt instructs model to think step by step before answering.

Complexity: ● ● ●

✓ Complex reasoning



Tree-of-Thought

Multiple reasoning paths

Model explores multiple branches before selecting best path.

Complexity: ● ● ● ●

✓ High-stakes decisions



Selection Guide

How to Choose

- **Zero-Shot:** Simple, clear tasks
- **Few-Shot:** Pattern learning needed
- **CoT:** Complex reasoning required
- **ToT:** Multiple options to evaluate

Best Use Case

Start with Zero-Shot for simple tasks, then progress to more complex techniques as needed.

Same Task, Four Ways — Compare Outputs

See how different prompting techniques affect the same business task



Business Task:

Evaluate a vendor proposal for cloud infrastructure services - analyze cost, security, scalability, and support quality



Zero-Shot

No examples given

Cost Analysis: \$2,500/month

Security: SOC 2 compliant

Scalability: 99.9% uptime

Support: 24/7 available

⚠ Limited detail

Quality: ●●●●●

3/5



Few-Shot

3 examples provided

Cost Analysis: \$2,500/month

Security: SOC 2, ISO 27001

Scalability: 99.9% uptime, auto-scaling

Support: 24/7, 2hr response time

✓ Good detail

Quality: ●●●●●

4/5



CoT

Step-by-step reasoning

1. **Cost:** \$2,500/month vs \$3,200 competitor

2. **Security:** SOC 2, ISO 27001, encryption

3. **Scalability:** 99.9% uptime, auto-scaling

4. **Support:** 24/7, 2hr response, dedicated rep

✓ Excellent detail

Quality: ●●●●●

5/5



ToT

Multi-path evaluation

Option A: \$2,500/month (Best value)

Option B: \$3,200/month (Premium)

Option C: \$1,800/month (Basic)

Recommendation: Option A

✓ Comprehensive

Quality: ●●●●●

5/5



Key Insight: Chain-of-Thought and Tree-of-Thought provide the most detailed analysis, while Few-Shot offers good balance for simple tasks.

🕒 3 min read



System vs User Prompts — Why the Distinction Matters

Understanding the two-layer architecture that makes agents reliable and safe

Two-Layer Architecture

● Active



System Prompt Layer

Constitution, rules, permissions, safety guardrails



Role Definition



Tool Permissions



Output Rules



Safety Guardrails

Escalation Rules



User Prompt Layer

Today's specific task and request



Current Task



Specific Request



Time Context



The Analogy

System prompt is like the employee handbook + job description.

User prompt is today's task assignment.



80% of agent reliability problems come from weak or missing system prompts — not from the user's questions.



Weak vs Strong System Prompt — Output Delta

See how system prompt quality dramatically affects output reliability and consistency

</> Output Comparison

● Same User Prompt, Different System Prompts

✗ Weak System Prompt

User: "Summarize the Q3 earnings report"

AI: "Here is a summary of the Q3 earnings report..."

- Generic response
- No specific details
- Inconsistent formatting
- Missing key metrics

⚠ Missing role definition and context

✓ Strong System Prompt

User: "Summarize the Q3 earnings report"

AI: "Q3 2024 Revenue: \$2.4M (+15% YoY)..."

- Structured output
- Key metrics included
- Consistent format
- Relevant context

✓ Properly engineered system prompt

📄 Support Agent Template

// **ROLE:** You are a professional customer support agent

// **CONTEXT:** Handle customer inquiries about products

// **TASK:** Provide helpful, accurate responses

// **FORMAT:** Friendly, professional tone

// **CONSTRAINTS:** Max 150 words, no jargon

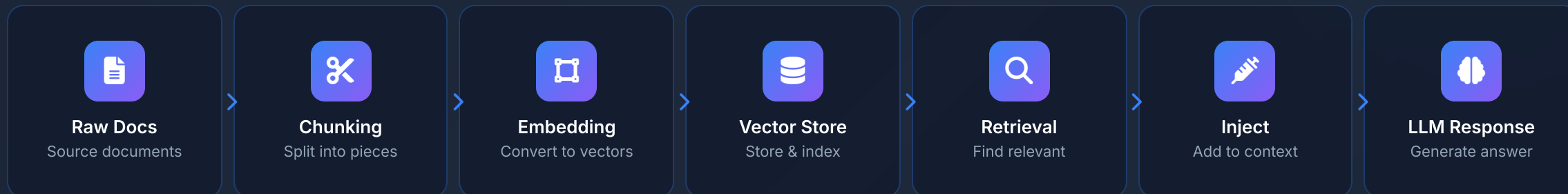
// **ESCALATION:** If issue cannot be resolved, escalate to supervisor

Context Engineering — Knowledge Pipeline

Design what information the agent has access to, in what form, at what moment

Context Engineering Pipeline

● Active



Why Context Engineering Matters

- An agent is only as good as its context**
Garbage in, hallucination out — quality retrieval reduces false information
- Embeddings explained**
Words converted to coordinates in a map of meaning — similar meanings cluster together
- Dynamic injection**
Agent decides at runtime which knowledge to retrieve based on current task

Key Design Principles

- Chunk Size**
Optimal 500-1000 tokens
- Metadata**
Tag for better retrieval
- Freshness**
Regular data updates
- Relevance**
Semantic similarity



Knowledge Base — Poor vs Well-Designed

See how proper chunking and metadata improve retrieval accuracy

Retrieval Quality Comparison

Poor

Good



Poor KB Design

Vague chunks, missing metadata, no context. Documents are too long, poorly structured, and lack proper tagging.

Retrieval: 42% accuracy



Well-Designed KB

Task-sized chunks, rich metadata, clear context. Documents are properly structured with relevant tags.

Retrieval: 94% accuracy



Chunking

Large documents split into **task-sized chunks** with proper overlap and context preservation.

Size: 500-1000 tokens



Metadata

Rich metadata including **source, date, category**, and relevance scoring.

Tags: 8+ per document



5 Design Rules

1

Chunk Size Matters

Keep chunks 500-1000 tokens with 20% overlap

2

Metadata Tagging

Add source, date, category, and relevance

3

Context Preservation

Maintain document structure and relationships

4

Freshness Control

Update knowledge base regularly

5

Quality Scoring

Implement relevance ranking for results



Managing Hallucination, Injection & Drift

Three critical failure modes and their defense mechanisms

Threat Map — Three Failure Zones

• Active Monitoring



Hallucination

Model generates false information confidently. No grounding, over-reliance on parametric knowledge.

Defense: RAG + Citations + Validation



Prompt Injection

Malicious input overrides system prompt instructions. No input sanitization.

Defense: Input validation + Role separation



Accuracy Drift

Outputs become less reliable over time as context grows or data goes stale.

Defense: Context management + Fresh data

Real-World Scenario



Financial Analyst Agent

Summarizing competitor Q3 earnings



Risk Points

Hallucination in numbers, injection in context



Defense Strategy

Source citations + validation layer



Prompt Iteration — Structured Debugging Flywheel

Continuous improvement cycle: Deploy → Observe → Identify → Diagnose → Refine → Redeploy

Improvement Flywheel

● Active



4-Step Debugging Framework

- 1 Reproduce**
Can you make the failure happen consistently?
- 2 Isolate**
Which component of the prompt caused it?
- 3 Hypothesize**
What assumption was wrong?
- 4 Test**
Change one variable at a time



Agent Built This Session — Prompt Coach Agent

Automated prompt engineering with diagnostic capabilities and improvement recommendations



Prompt Coach Agent

AI-Powered Prompt Engineering Assistant

🎯 Purpose

Takes any weak or vague prompt submitted by a learner and returns a fully engineered version with before/after comparison and diagnosis of what was missing.

Architecture Flow

● Active



Raw Prompt
Learner input



System Prompt
Expert framework



LLM
Processing



Structured Output
Diagnosis + Improved

Inputs

Any raw prompt

Outputs

4 components

Techniques

4 methods

Examples

20+ samples



Deployment

- ✓ Slack integration available
- ✓ Web interface for testing
- ✓ API endpoint for automation

Knowledge Base

20+ examples

5-component framework

4 technique descriptions

HR examples

Sales examples

Finance examples

Support examples



Recap & What's Next

Summary of key concepts and bridge to the next topic



5-Component Blueprint

Role, Context, Task, Format, Constraints



4 Techniques

Zero to Tree-of-Thought



System Constitution

Agent identity & rules



Context Design

Design what agents know



3 Failure Modes

Hallucination, Injection, Drift



Iteration Flywheel

Structured debugging



What's Next: RAG

You now know how to write prompts that make agents reliable. Next — you'll give your agent a brain built from your own data. **That's RAG.**

Bridge: From prompt engineering to context engineering with your private knowledge base.



Reflection Prompt

Choose one agent you want to build after this course. Write its system prompt using the 5-component framework right now.



Role, Context, Task, Format, Constraints